System Design Interview Tool — Final Project Roadmap

A full-stack tool where users diagram system architectures and get AI-driven critique, grounded in deterministic checks + RAG, with a live "Interview Mode."

Target timeline: 7–8 months (solo, out of a 1-year window)

---

1. Guiding Sentence

> **Build the smallest system that demonstrates the architectural concept correctly.**

Whenever unsure whether to add or cut something, this is the tiebreaker.

---

2. Non-Functional Requirements (NFR)

- CRUD endpoints remain responsive under normal load
- Cached AI responses typically <200ms
- WebSocket status updates feel near real-time
- Zero data loss on normal save operations
- Support at least 100 concurrent simulated users (load-tested, not assumed)
- One-command Docker Compose setup
- 95%+ successful request rate under load test

---

3. Engineering Principles

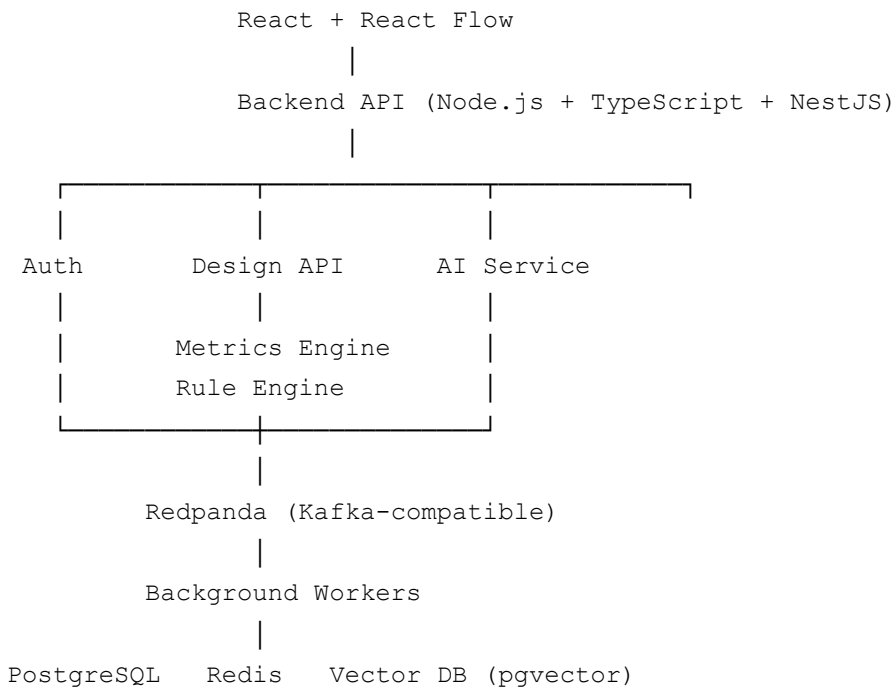
1. **Every phase ends with a working demo.** No "80% complete." If a feature isn't demoable, it isn't done.
2. **Every technology must solve a real problem.**

- Redis → repeated AI requests are expensive
- Kafka → AI analysis is asynchronous
- Vector DB → RAG needs semantic retrieval
- If a technology doesn't remove a bottleneck or enable a feature, don't add it.

3. **Deterministic before AI.** Metrics Engine → Rule Engine → RAG → LLM reasoning. More reliable, easier to explain, less hallucination.
4. **Measure, don't assume.** Latency, cache hit rate, queue time, throughput, memory — never claim a number you didn't measure.
5. **Optimize only after it works.** Correctness → performance → polish, in that order.

---

4. Final Architecture



Deliberately excluded: separate API Gateway service (use middleware/BFF instead), full RBAC (use resource ownership + optional `is_admin`), CRDT collaborative editing, split AI microservices, Kubernetes (unless everything else finishes early), Grafana (same condition).

Standardized AI Pipeline (defined once, referenced everywhere)

```
React Flow JSON (versioned, e.g. `"version": 1`)
```

```
  ↓
Domain Model (normalized, UI-independent representation)
  ↓
Metrics Engine
  ↓
Rule Engine
  ↓
RAG Retrieval
  ↓
Prompt Builder
  ↓
LLM
  ↓
Critique Formatter
  ↓
WebSocket Stream
```

The AI pipeline never sees raw React Flow JSON directly — only the normalized domain model. This keeps the backend independent of frontend-specific data shapes.

Metrics Engine Output Contract

Every later phase (Rule Engine, RAG prompt, Interview Mode scoring) consumes this exact shape:

```
{
  "services": 8,
  "databases": 2,
  "caches": 1,
  "queues": 1,
  "spof": ["Primary Database"],
  "fanOut": 3.2,
  "redundancyScore": 82
}
```

Failure Handling

If the LLM call times out or fails:

- Return Rule Engine results only (already computed upstream in the pipeline)
- Show the user: *"AI analysis unavailable. Showing deterministic analysis."*
- Never let an LLM failure produce a blank or broken response — graceful degradation is part of the spec, not an afterthought.

Frozen Tech Stack

Once Phase 1 starts, this list does not change unless you hit a genuine blocker:

React · TypeScript · NestJS · PostgreSQL · Redis · pgvector · Redpanda · Docker

No mid-project framework or model swaps for novelty. Swapping the LLM provider via the `callLLM()` abstraction is fine — that's what it's for.

Four Logical Modules

A mental model for organizing folders and responsibilities (not separate services):

- **Editor** — React Flow, diagram storage, versioning
- **Analysis** — Metrics Engine, Rule Engine
- **AI** — RAG, Prompt Builder, LLM, Formatter
- **Interview** — question generation, conversation state, scoring

Keeping these boundaries clear in code makes each one independently testable, without turning any of them into an actual microservice.

---

5. Free Stack (build this for ~$0)

Layer	Choice	Why
Frontend hosting	Vercel / Netlify free tier	Standard, zero-cost
Backend hosting	Render / Railway free tier	Sufficient for demo traffic
PostgreSQL	Supabase / Neon / Railway	Free tier covers this project
Vector DB	pgvector (extension on same Postgres)	No separate service needed
Redis	Upstash free tier	Serverless, generous quota
Queue	Redpanda (self-hosted in Docker), Kafka-compatible	Same tool locally and deployed, free
CI/CD	GitHub Actions free tier	Generous for solo repos

LLM (dev/default)	**Gemini free tier** (or Groq for fast streaming)	Best free quality/speed for this use case
LLM (local fallback)	Ollama + Llama 3.1 8B / Gemma 2 9B	Only if you hit free-tier rate limits during heavy test days
LLM (final demo only)	Claude or GPT-4 class, paid	~$5–10 total, just for polish on your demo recording

AI service is provider-agnostic — one `callLLM(prompt)` interface, so swapping Gemini → Groq → Claude is a one-line config change, not a refactor.

---

6. Phase Plan

Phase 1 — Foundations

- React + TypeScript, React Flow canvas (drag/drop, connect, zoom/pan), Tailwind + shadcn/ui
- Docker Compose skeleton, GitHub Actions CI (lint + test)
- JWT auth + refresh tokens
- **Resource ownership** (`user_id` owns projects) + optional `is_admin` flag — *not* full RBAC
- PostgreSQL: users, projects, diagrams
- Swagger/OpenAPI docs, health checks, structured logging
- ✅ Milestone: sign up → create project → draw architecture → save diagram

Phase 2 — Core Product

- Design service: CRUD, autosave, **event-based version history**, optimistic updates
- UI: node config, labels, delete, duplicate, undo/redo
- **Metrics Engine** ⭐: compute service count, DB count, cache count, SPOFs, load balancers, queues, cyclic dependencies, fan-out, missing redundancy → feed into AI prompt
- AI v1: diagram JSON + metrics → prompt → LLM → critique (no RAG yet)

- ✅ Milestone: draw architecture → AI gives useful critique grounded in real stats

Phase 3 — Performance Layer

- Redis: response caching, sessions, rate limiting
- WebSockets: streamed status updates ("Checking redundancy... Checking cache strategy...")
- Write 15–20 knowledge base docs: load balancers, CDN, Redis, Kafka, sharding, replication, CAP theorem, CQRS, event sourcing, rate limiting, indexing, microservices, API gateway patterns, consistency models, failover
- ✅ Milestone: AI feedback streams live, feels fast on repeat queries

Phase 3.5 — Rule Engine

- Deterministic checks before the LLM runs, e.g.:

```
if (dbCount == 1) warn("Single Point of Failure")
if (cacheCount == 0 && traffic > 10000) warn("Missing Cache")
if (services > 8 && queue == 0) warn("Consider async communication")
```

- Pipeline becomes: `Diagram → Metrics Engine → Rule Engine → AI (RAG) → Final Report`
- Keep to a handful of high-value rules, not dozens
- ✅ Milestone: consistent, fast, low-hallucination baseline feedback even before AI runs

Phase 4 — RAG

- Embed knowledge base into pgvector
- Pipeline: question → embed → vector search → relevant docs → prompt → LLM
- Add tool calling (e.g., AI fetches live diagram metrics as a function call)
- ✅ Milestone: critiques cite retrieved patterns, not just LLM guessing — noticeably stronger than Phase 2

Phase 5 — Async Architecture

- Redpanda: queue **only real work** (AI analysis, simulation jobs)
- Background workers consume jobs, push results back via WebSocket

- Local load testing (k6 / autocannon) for real throughput numbers
- Rule: if a request takes >~1s or doesn't need an immediate response → background job; otherwise stay synchronous
- ✅ Milestone: demoable behavior under concurrent load — your strongest interview story

Phase 6 — Interview Mode ⭐⭐⭐⭐⭐

- AI acts as interviewer: "Design WhatsApp" → user draws → AI asks "Why Redis? What if Postgres fails? How does this scale to 50M DAU?" → user updates diagram → AI scores (e.g., 7.8/10) with strengths/weaknesses/suggestions
- **Keep v1 simple:** prompt → user draws → AI asks 5 questions → user answers → final score. No voice, avatars, or timers until everything else is complete.
- **Scoring rubric, fixed in advance** so scores stay consistent across runs:

Category	Weight
Availability	25%
Scalability	25%
Reliability	20%
Performance	15%
Security	10%
Cost awareness	5%

- Your killer, most memorable demo feature
- ✅ Milestone: full interactive interview loop, not just a static analyzer

Phase 7 — Simulation Engine *(nice-to-have, only if time permits)*

- User sets load params (users, req/s, cache, replication)
- Rule-based simulator predicts latency, bottlenecks, cache hit estimate, DB saturation, queue backlog
- AI explains the output in plain language

- Believable estimates with stated assumptions > false precision
- ✅ Milestone: draw → simulate → AI explains what breaks and why

Phase 8 — Polish, Deployment, Benchmarking

- Deploy to a real URL (not localhost)
- README: architecture, tech decisions, trade-offs, future work
- Real performance testing (k6/autocannon) — **only report measured numbers**
- Demo video/GIF
- Final resume bullets with real numbers

---

7. MVP Boundaries

MVP (must have): Authentication · Diagram editor · Save/load projects · Metrics engine · Rule engine · AI critique · RAG · Redpanda for AI analysis · Interview Mode · Deployment

Nice to have: Simulation engine · Report export · Performance dashboard · Kubernetes

Skip unless everything else is finished: Multi-user collaboration · Advanced autoscaling · Complex admin features

(CRDT/collaborative editing is not on this conditional list — it's permanently excluded, see Section 4.)

If you fall behind: cut Kubernetes first, then simulation engine. Never cut the deploy/polish phase — that's what makes the project resume-ready rather than just code-complete.

---

8. Architecture Decision Records (ADRs)

Keep a `docs/adr/` folder. One page per significant decision, written **as you make it**, not retroactively. Template:

```
ADR-00X: [Decision title]

Context
[What problem prompted this decision]

Decision
```

```
[What you chose]

Alternatives considered
- Option A
- Option B

Consequences
+ [benefit]
+ [benefit]
- [trade-off / added complexity]
```

Suggested ADRs to write as you go:

- Why resource ownership instead of full RBAC
- Why Redpanda instead of synchronous AI calls
- Why a Rule Engine before the LLM
- Why Gemini/Groq free tier for dev, with provider abstraction
- Why pgvector instead of a separate vector DB service
- Why middleware/BFF instead of a dedicated API Gateway service

These are excellent interview material — they show reasoning, not just tool usage.

---

9. Testing Strategy

Doesn’t need to be exhaustive — needs to show you think about quality.

- **Unit tests:** Metrics Engine, Rule Engine
- **Integration tests:** Auth, diagram saving, AI pipeline
- **End-to-end:** login → draw diagram → AI critique → Interview Mode

10. Security Considerations

- Password hashing: Argon2 or bcrypt
- JWT expiration + refresh token rotation
- Input validation on all API endpoints
- API rate limiting (already have Redis for this)

- CORS configuration
- Secrets via environment variables, never committed
- Parameterized SQL / ORM to prevent injection

11. Version Milestones

Tag releases in Git as you complete each phase:

```
v0.1  Authentication
v0.2  Diagram Editor
v0.3  Metrics Engine
v0.4  AI Critique (Rule Engine + basic LLM)
v0.5  Redis + WebSockets
v0.6  RAG
v0.7  Redpanda + Background Workers
v0.8  Interview Mode
v0.9  Simulation Engine (if time permits)
v1.0  Production Release (deployed, documented, benchmarked)
```

Clean release history = easy to demonstrate progress and talk through in interviews.

12. Resume-Worthy Metrics to Aim For

Targets are already defined in Section 2 (NFRs). Only claim numbers you've actually measured. Two additional metrics worth tracking specifically for your resume/demo, beyond the NFRs:

- Cache hit rate on repeated analysis requests (target: 70–90%)
- Queue/worker throughput under simulated load

Sample resume bullet (draft for Phase 8):

> Built an AI-powered system design interview platform enabling users to design distributed systems visually and receive deterministic + RAG-enhanced architectural feedback. Implemented a React Flow editor, metrics/rule engine, Redpanda-backed async analysis pipeline, and an interactive AI interview mode with real-time WebSocket streaming.

13. Expect These to Take Longer Than They Look

Budget extra buffer for:

- Designing a domain model that’s genuinely decoupled from React Flow’s JSON shape
- Writing Rule Engine checks that catch real issues without excessive false positives
- Curating a RAG knowledge base where quality beats quantity
- Prompt engineering Interview Mode so questions feel coherent and build on prior answers, not random